Residência Tecnológica — UFPE/CIn

Remote Firmware Flasher

Manual do Usuário

Sistemas Operacionais de Tempo Real

Versão	2.0
Data	Março de 2026
Repositório	`renatosfagundes/remote-flasher`
Plataforma	Windows 10/11

Gerado por `manual.tex` — Compilado com Tectonic

Sumário

1 Introdução **4**
1.1 Funcionalidades 4

2 Instalação **4**
2.1 Executável (recomendado) 4
2.2 Código-fonte 4
2.3 Configurando as credenciais (`secrets.py`) 4

3 Configuração do Ambiente de Desenvolvimento **5**
3.1 O que é instalado automaticamente 5
3.2 Executando o Setup 5
3.3 Verificação manual 5
3.4 Estrutura de diretórios 6

4 Primeira Execução **6**

5 Conectando à VPN **6**

6 Gravando Firmware (Flash) **8**

7 Configuração de Rede CAN **9**
7.1 Topologia visual 9
7.2 Selecionando a rede 10
7.3 Presets rápidos 10

8 Terminal Serial **11**
8.1 Layout adaptativo 11
8.2 Enviando comandos 13
8.3 Autoscroll 13
8.4 Exclusão de portas 13
8.5 Upload do serialterm.py 13
8.6 Virtual I/O — Controles integrados 14

9 Terminal SSH e Upload de Arquivos **15**
9.1 Executar comandos 15
9.2 Upload de arquivos e pastas (SFTP) 15

10 Câmera ao Vivo **16**

11 Integração com Notepad++ **16**
11.1 Plugins necessários 17
11.2 Exemplo: Compilar e gravar um projeto Trampoline 17
11.3 Máscaras de destaque para erros 18

12 Integração com VSCode **19**
12.1 Extensões recomendadas 19

12.2 IntelliSense para AVR . . . 19
12.3 Tarefas de build (tasks.json) . . . 20
12.4 Configurações recomendadas . . . 21

13 Guia de Desenvolvimento — Criando seu Projeto 21
13.1 Passo a passo . . . 21

14 Referência Rápida — Arquivo OIL 21
14.1 Estrutura básica . . . 21
14.2 TASK — Declaração de tarefa . . . 22
14.3 ALARM — Ativação periódica . . . 22
14.4 RESOURCE — Exclusão mútua . . . 23

15 Referência Rápida — Comunicação CAN 23
15.1 Configuração no arquivo OIL . . . 23
15.2 Inicialização . . . 23
15.3 Enviar mensagem . . . 23
15.4 Receber mensagem . . . 24
15.5 Encapsular/Desencapsular sinais . . . 24

16 Referência Rápida — Board.h (Pinos da Placa) 24

17 Referência Rápida — Variáveis Compartilhadas entre Tasks 25

18 Exemplos do Repositório 26

19 Exemplo Prático: LED Controlado por Alarme 26
19.1 Estrutura do projeto . . . 26
19.2 Arquivo OIL . . . 27
19.3 Código-fonte . . . 28
19.4 Compilar e gravar . . . 28

20 Exemplo Prático: Comunicação CAN com RTOS 29
20.1 ECU Transmissora (can_pot_tx) . . . 29
20.2 ECU Receptora (can_pot_rx) . . . 30

21 Virtual I/O — Controles Remotos 31
21.1 Protocolo . . . 31
21.2 LEDs Virtuais (saídas) . . . 31
21.3 Biblioteca Arduino: VirtualIO . . . 31
21.4 Exemplo 1: LED controlado por botão e brilho por potenciômetro . . . 32
21.5 Exemplo 2: CAN com controles virtuais . . . 34
21.6 Exemplo 3: Leitura mista (virtual + real) . . . 35

22 Configuração dos PCs do Laboratório 35

23 Estrutura do Projeto 36

24 Solução de Problemas 36
24.1 “make.py” abre diálogo de seleção de programa . . . 36

24.2 “goil” não é reconhecido como comando 36
24.3 VPN não conecta 36
24.4 Erro ao gravar firmware 37
24.5 Terminal serial: linhas quebradas ou caracteres estranhos 37
24.6 Setup: “Python interpreter not found” 37
24.7 Setup: download falha com erro SSL 37

1. Introdução

O **Remote Firmware Flasher** é uma aplicação desktop (PySide6) que permite programar placas Arduino remotamente, através de conexão VPN e SSH. Foi desenvolvido para a Residência Tecnológica do CIn/UFPE, permitindo que alunos gravem firmware nas placas do laboratório sem precisar estar fisicamente presente.

1.1 Funcionalidades

- Conexão VPN (SSTP) diretamente pela aplicação
- Upload e gravação de firmware (`.hex`) via SCP + `avrdude`
- Reset das placas via scripts PowerShell remotos
- Até 4 terminais serial simultâneos com envio de comandos
- Terminal SSH para executar comandos no PC remoto
- Upload de arquivos e pastas via SFTP com barra de progresso
- Câmera ao vivo para visualizar as placas
- Configuração automática do ambiente de desenvolvimento (Setup)
- Guias de integração com Notepad++ e VSCode

2. Instalação

2.1 Executável (recomendado)

Basta baixar o arquivo `RemoteFlasher.exe` e executar. Não é necessário instalar Python ou qualquer dependência.

2.2 Código-fonte

Para executar a partir do código-fonte:

```
git clone https://github.com/renatosfagundes/remote-flasher.git
cd remote-flasher
pip install -r requirements.txt
cp secrets.example.py secrets.py # editar com as credenciais do lab
python src/main.py
```

Dependências: Python 3.9+, PySide6, paramiko, requests.

2.3 Configurando as credenciais (`secrets.py`)

O arquivo `secrets.py` contém os endereços IP, usuários e senhas SSH dos PCs do laboratório. Esse arquivo **não é incluído no repositório** por segurança. Para obtê-lo, entre em contato com **Davi** ou **Renato**.

3. Configuração do Ambiente de Desenvolvimento

A aba **Setup** do aplicativo automatiza toda a configuração do ambiente necessário para compilar e gravar firmware usando o Trampoline RTOS.

3.1 O que é instalado automaticamente

Componente	Descrição
Arduino CLI	Ferramenta de linha de comando para compilar e gravar firmware Arduino
AVR Toolchain	Compilador GCC para microcontroladores AVR (avr-gcc, avr-objcopy, etc.)
Trampoline RTOS	Sistema operacional de tempo real compatível com OSEK/AUTOSAR
goil	Compilador OIL (OSEK Implementation Language) do Trampoline
Arduino Core	Bibliotecas do Arduino para uso com Trampoline
Template Nano	Template de configuração do Trampoline para Arduino Nano
MCP_CAN	Biblioteca para comunicação CAN via módulo MCP2515

3.2 Executando o Setup

1. Abra a aba **Setup**
2. Os indicadores mostram o status de cada componente (verde = instalado, vermelho = ausente)
3. Clique em **"Run Full Setup"** para instalar tudo automaticamente
4. Acompanhe o progresso no log à esquerda
5. Após a conclusão, abra um **novo terminal** para que as variáveis PATH sejam reconhecidas

Importante

O setup requer conexão com a internet para baixar os componentes. O processo pode levar alguns minutos na primeira execução. Execuções subsequentes são rápidas pois detectam o que já está instalado.

3.3 Verificação manual

Após o setup, abra um novo terminal e verifique:

```
arduino-cli version
avr-gcc -version
goil -version
```

3.4 Estrutura de diretórios

Todos os componentes são instalados em `C:\ESA`:

```
C:\ESA\
arduino_cli\bin\    # Arduino CLI
avr_tools\            # AVR GCC Toolchain + goil
trampoline\          # Trampoline RTOS
examples\avr\    # Exemplos (arduinoUno, arduinoNano)
goil\templates\  # Templates de configuração
opt\devel\      # Diretório de desenvolvimento
```

4. Primeira Execução

Na primeira vez que você abrir o aplicativo, uma janela de configuração será exibida solicitando:

- **Seu nome** — usado para criar sua pasta no PC remoto.
- **Pasta remota** — preenchida automaticamente como `c:\2026\<seu_nome>`.

Configurações são salvas em `%APPDATA%\RemoteFlasher\settings.json`.

5. Conectando à VPN

A aba **VPN** permite conectar-se à rede do laboratório:

1. Insira seu usuário e senha do CIn
2. (Opcional) Marque “Remember me” para salvar as credenciais
3. Clique em “Connect VPN”
4. Aguarde o indicador ficar verde

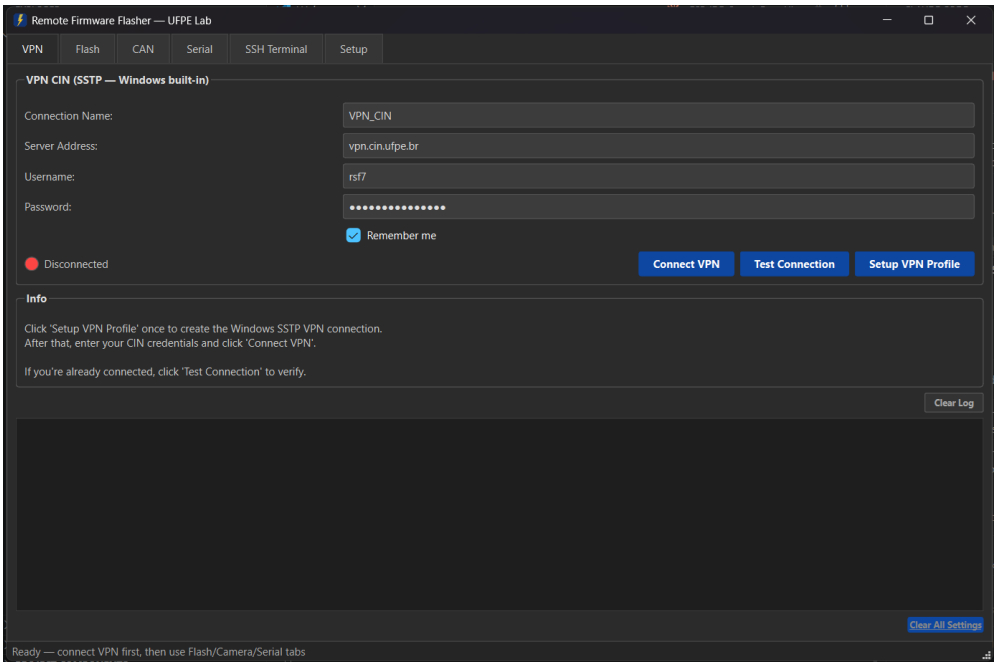

Figura 1: Aba VPN antes da conexão — indicador vermelho, campos preenchidos.

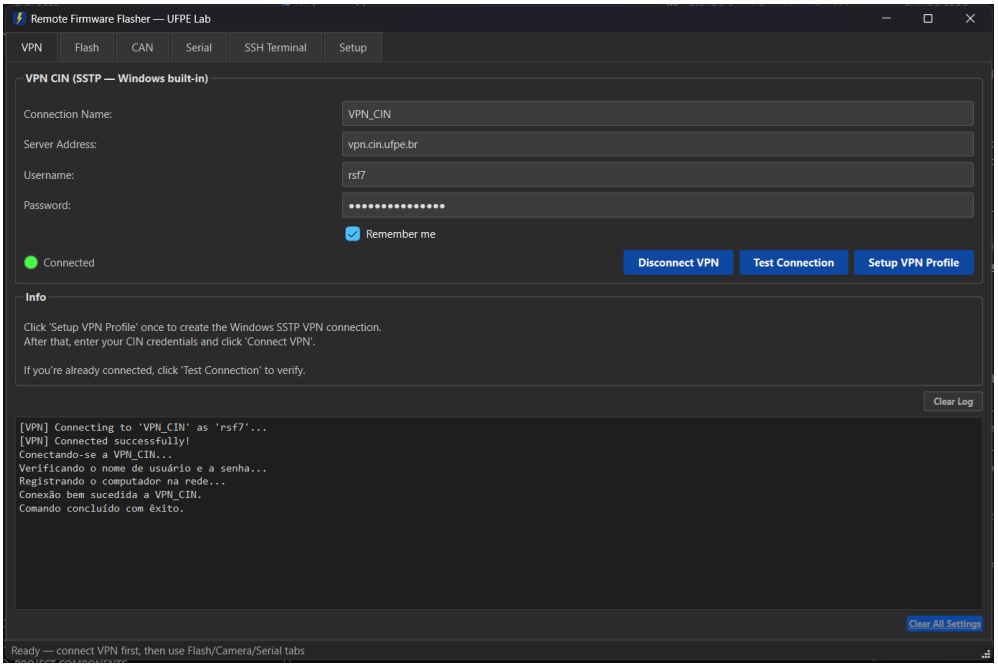

Figura 2: VPN conectada — indicador verde, log de conexão bem sucedida.

Ao fechar o aplicativo com a VPN conectada, um diálogo pergunta se deseja desconectar ou manter a conexão ativa:

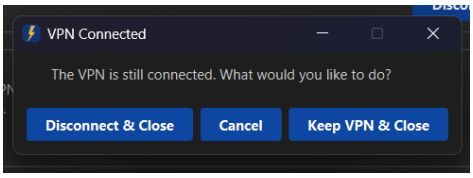

Figura 3: Diálogo exibido ao fechar o aplicativo com VPN ativa.

Nota

Se a VPN ainda não estiver configurada no Windows, clique em "Setup VPN Profile" primeiro. O aplicativo também cria o perfil automaticamente caso ele não exista ao clicar em "Connect VPN".

6. Gravando Firmware (Flash)

A aba **Flash** permite enviar e gravar firmware nas placas.

1. Selecione o PC de destino (ex: PC 217)
2. Selecione a placa (ex: Placa 01) e a porta ECU
3. Clique em "Browse" e selecione seu arquivo `.hex`
4. Use "Upload + Reset + Flash" para executar tudo de uma vez

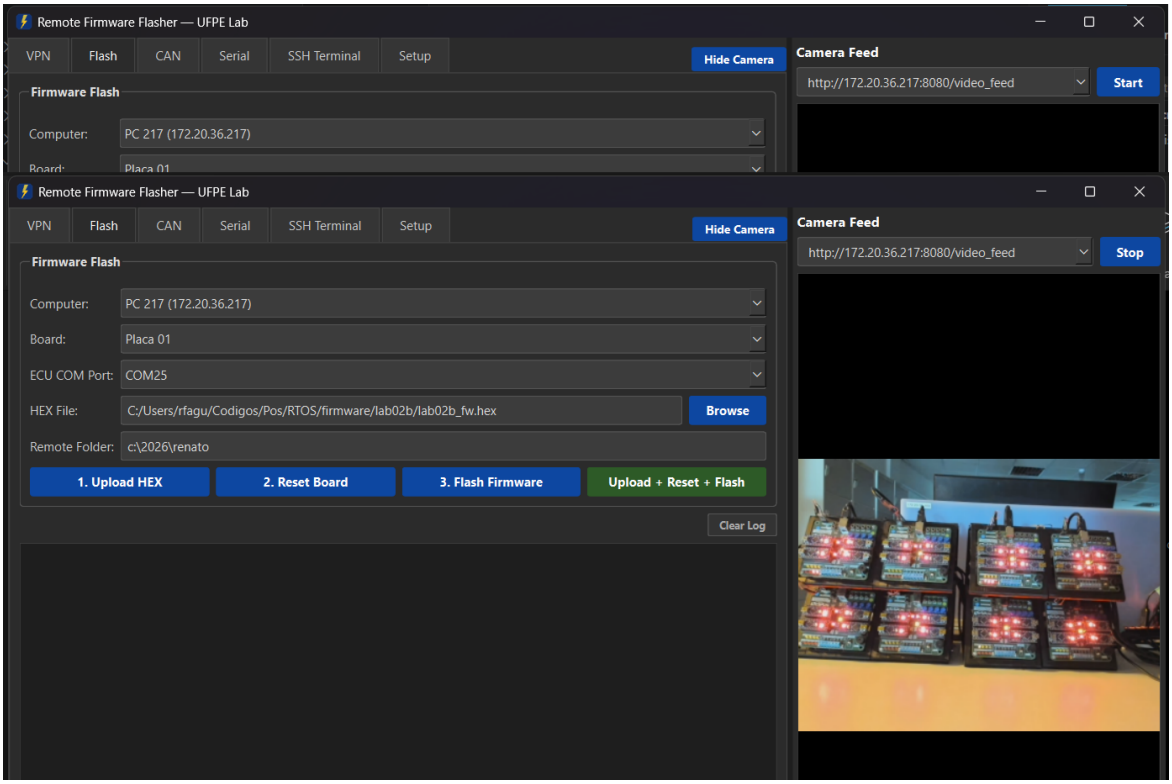

Figura 4: Aba Flash com arquivo selecionado e câmera ao vivo exibindo as placas.

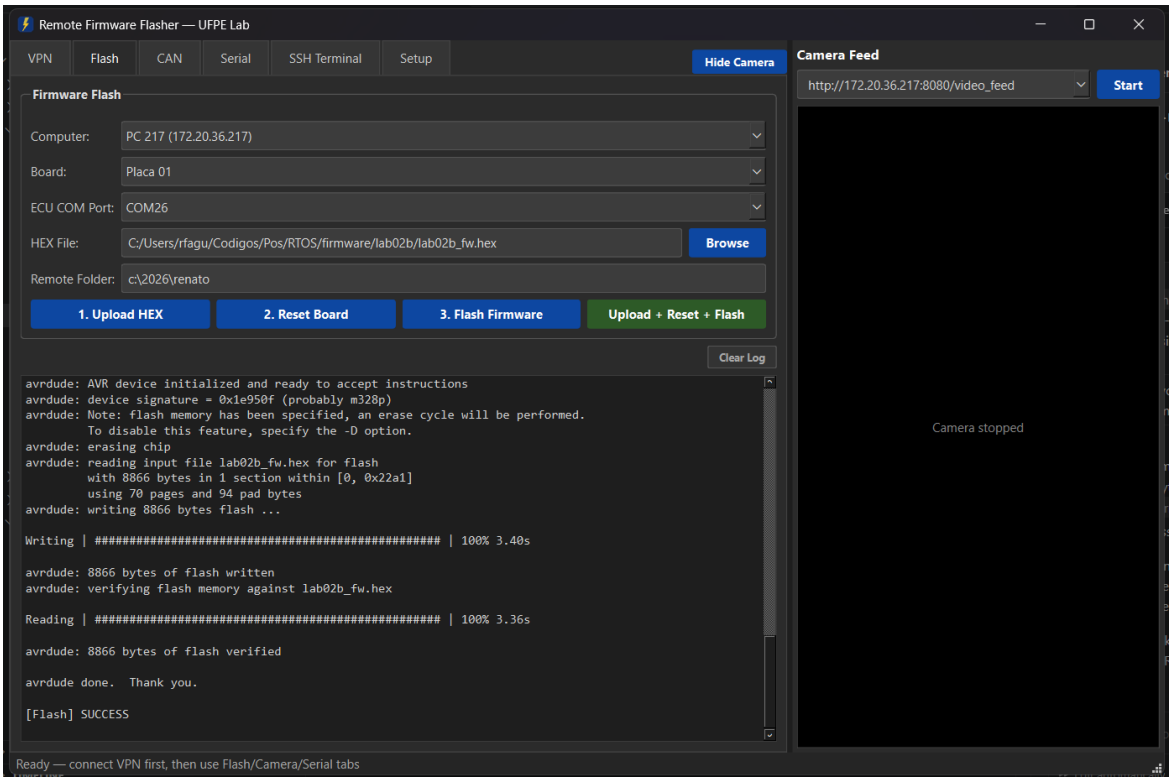

Figura 5: Flash concluído — saída do avrdude em tempo real com progresso de escrita e verificação.

7. Configuração de Rede CAN

A aba **CAN** permite configurar em qual barramento CAN (CAN1 ou CAN2) cada placa está conectada. A placa ANEB possui duas redes CAN independentes, e cada placa pode ser comutada entre elas via comando AT no microcontrolador de controle.

7.1 Topologia visual

A parte superior da aba mostra um diagrama em tempo real com dois barramentos (CAN 1 em verde, CAN 2 em laranja) e as placas conectadas a cada um. Placas sem seletor CAN aparecem em cinza com "?" entre os dois barramentos.

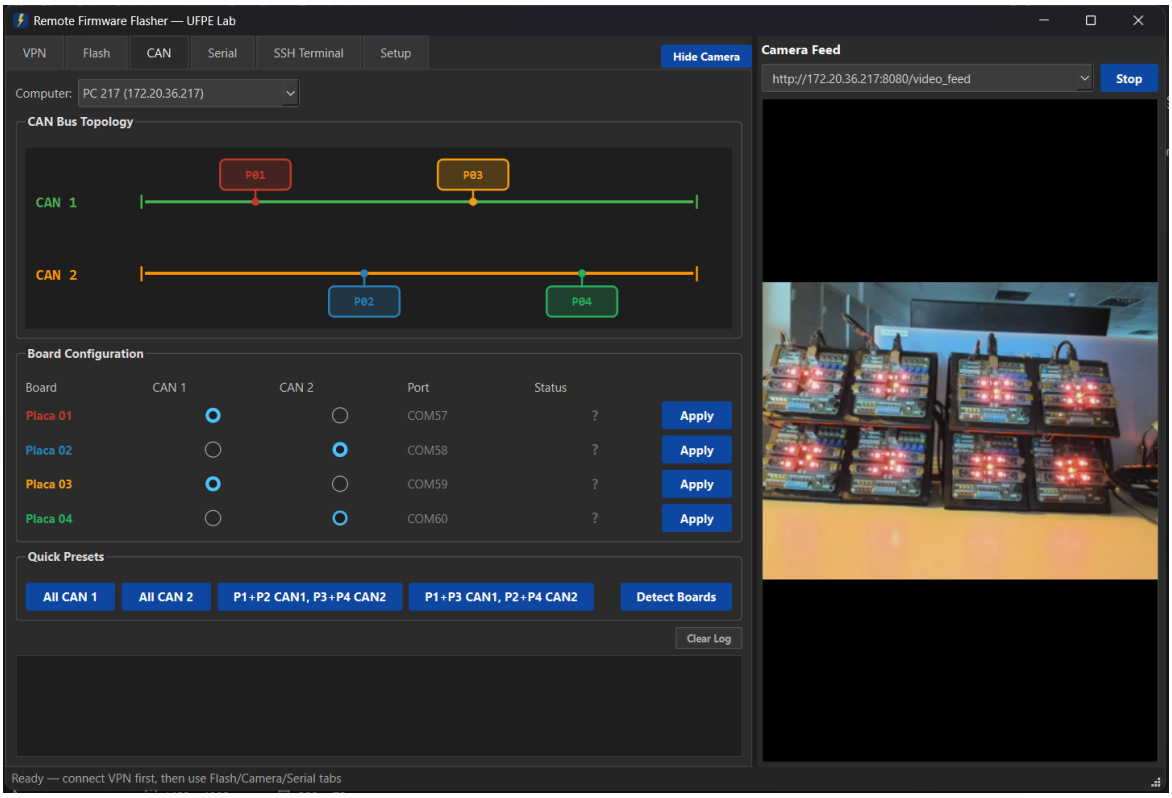

Figura 6: Topologia CAN — Placas 01 e 02 em CAN 1, Placas 03 e 04 em CAN 2.

7.2 Selecionando a rede

1. Selecione o PC no menu **Computer**
2. Para cada placa, selecione **CAN 1** ou **CAN 2**
3. Clique em **Apply** para enviar o comando de comutação
4. O log confirma com `MCU1 OK`

7.3 Presets rápidos

Botões de configuração rápida permitem colocar todas as placas no mesmo barramento ou dividir em grupos com um clique. O botão **Detect Boards** consulta a identidade e firmware de cada placa.

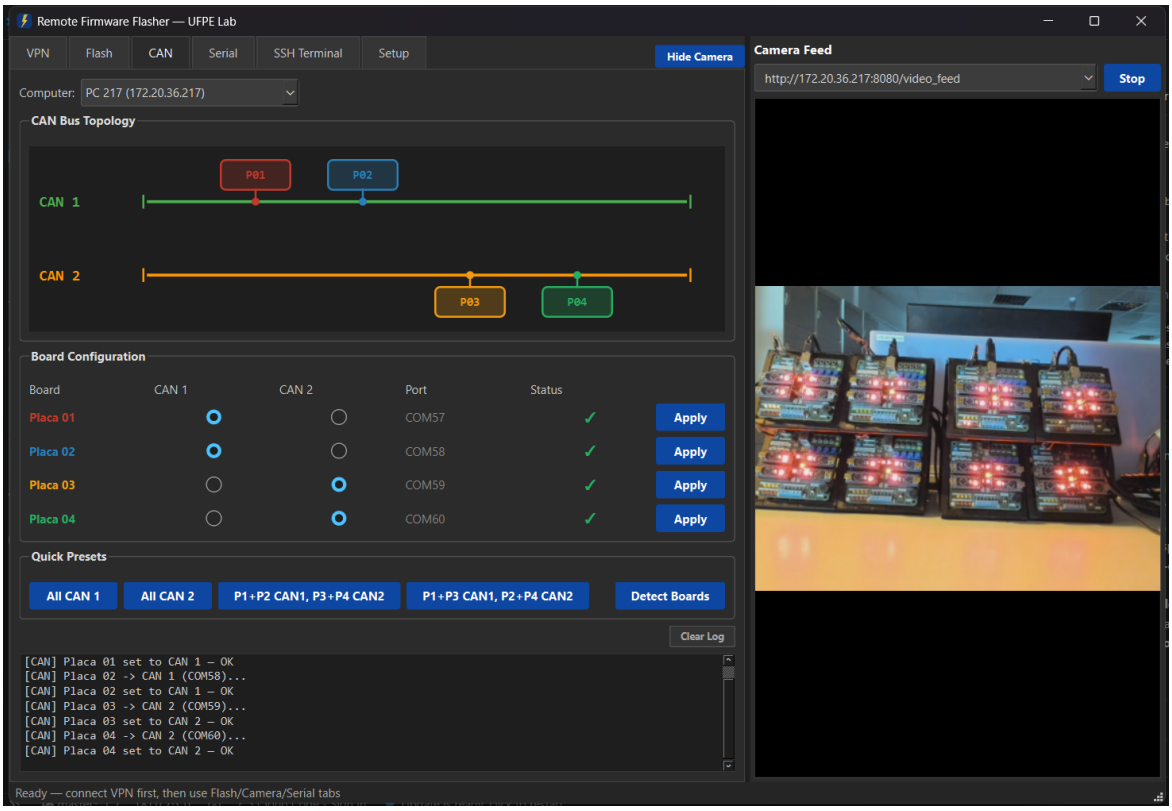

Figura 7: Todas as placas configuradas — status verde confirmando a aplicação.

Nota

Para que duas ECUs se comuniquem via CAN, ambas devem estar no **mesmo barramento**. Se a comunicação não funciona, verifique se ambas estão em CAN 1 (ou ambas em CAN 2).

8. Terminal Serial

A aba **Serial** permite monitorar até 4 conexões seriais simultaneamente e enviar comandos.

8.1 Layout adaptativo

- **1 conexão:** painel único, área completa
- **2 conexões:** lado a lado (divisor horizontal)
- **3 conexões:** 2 em cima + 1 embaixo
- **4 conexões:** grade 2×2

Clique em **"+ Add Serial"** para adicionar uma nova conexão (máximo 4). Cada painel pode ser fechado pelo botão **X** (que desconecta automaticamente).

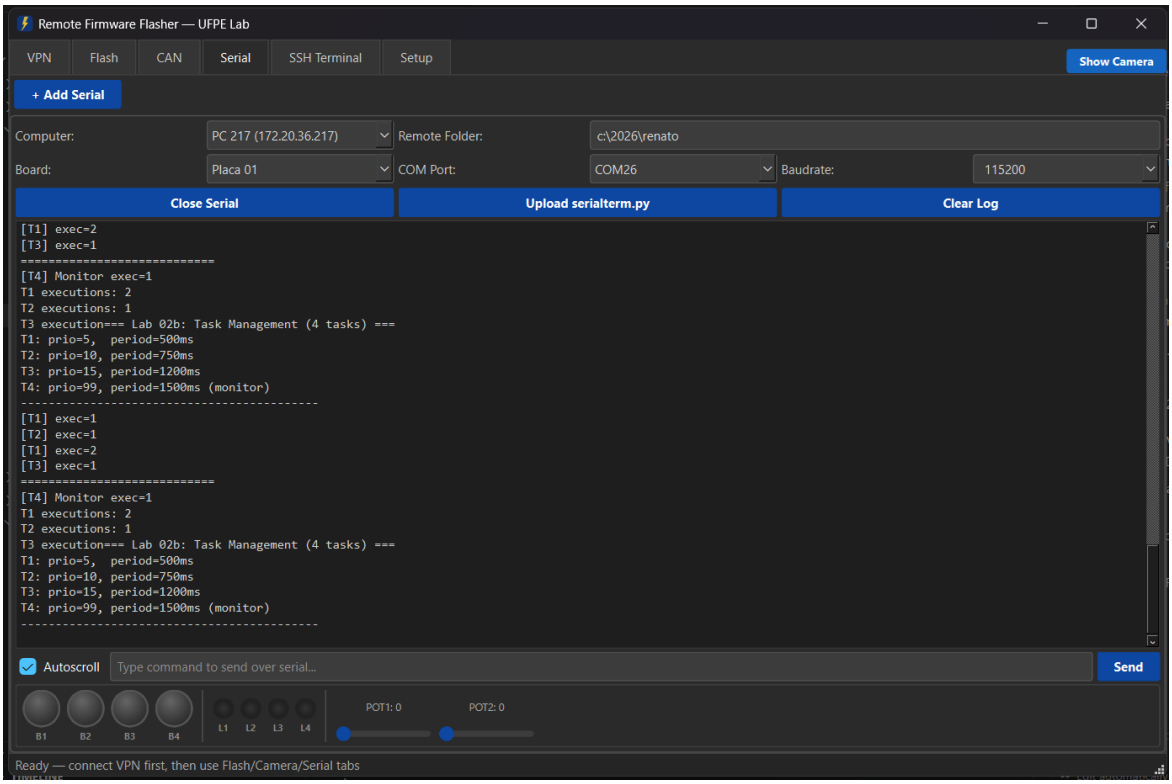

Figura 8: Um painel serial — saída de tarefas RTOS com Virtual I/O na parte inferior.

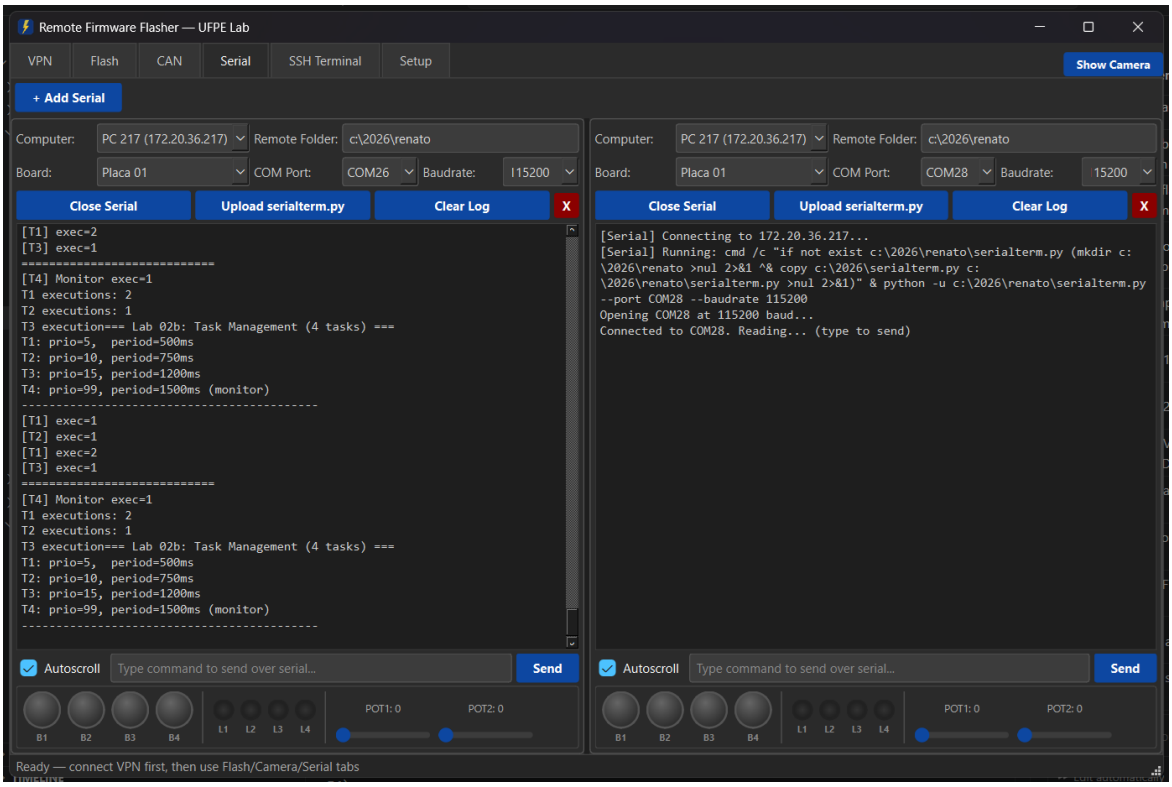

Figura 9: Dois painéis lado a lado — TX e RX monitorados simultaneamente.

8.2 Enviando comandos

Cada painel possui um campo de texto na parte inferior para enviar comandos pela porta serial. Digite o comando e pressione **Enter** ou clique em **Send**. O campo só fica ativo quando a conexão está aberta.

8.3 Autoscroll

Desmarque a caixa **"Autoscroll"** para congelar a posição do scroll enquanto os dados continuam chegando — útil para analisar mensagens CAN sem perder a posição.

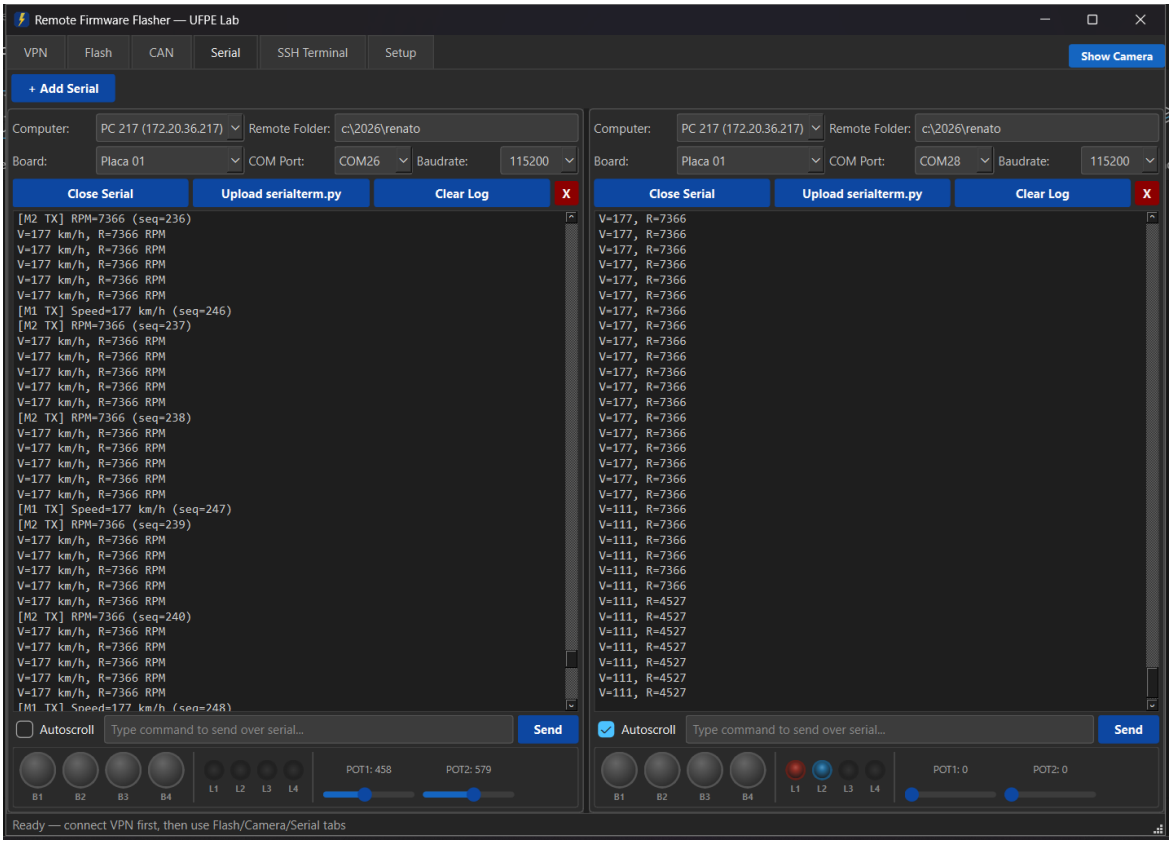

Figura 10: Autoscroll desativado no painel esquerdo enquanto dados CAN com sequenciador continuam chegando.

8.4 Exclusão de portas

Quando um painel está conectado a uma porta COM, essa porta não aparece nos menus dos outros painéis, evitando conflitos.

8.5 Upload do serialterm.py

Na primeira utilização, clique em **"Upload serialterm.py"** em qualquer painel serial **antes** de conectar. Isso envia a versão bidirecional do script para o PC remoto. Só é necessário fazer isso uma vez — o arquivo é reutilizado por todas as conexões do mesmo PC.

Importante

Se o script remoto for antigo (versão somente leitura), os controles Virtual I/O não funcionarão. Sempre faça o upload ao usar uma versão nova do Remote Flasher.

8.6 Virtual I/O — Controles integrados

Cada painel serial possui um painel de **Virtual I/O** na parte inferior com:

- **4 botões** (B1–B4) — redondo, 3D, com cores distintas. Clique para pressionar/soltar.
- **4 LEDs** (L1–L4) — indicadores que o firmware controla remotamente. O brilho varia de 0 (apagado) a 255 (máximo).
- **2 potenciômetros** (POT1, POT2) — sliders de 0 a 1023 que enviam valores ao firmware.

Os controles ficam ativos somente quando a conexão serial está aberta. Ao desconectar, todos voltam ao estado inicial.

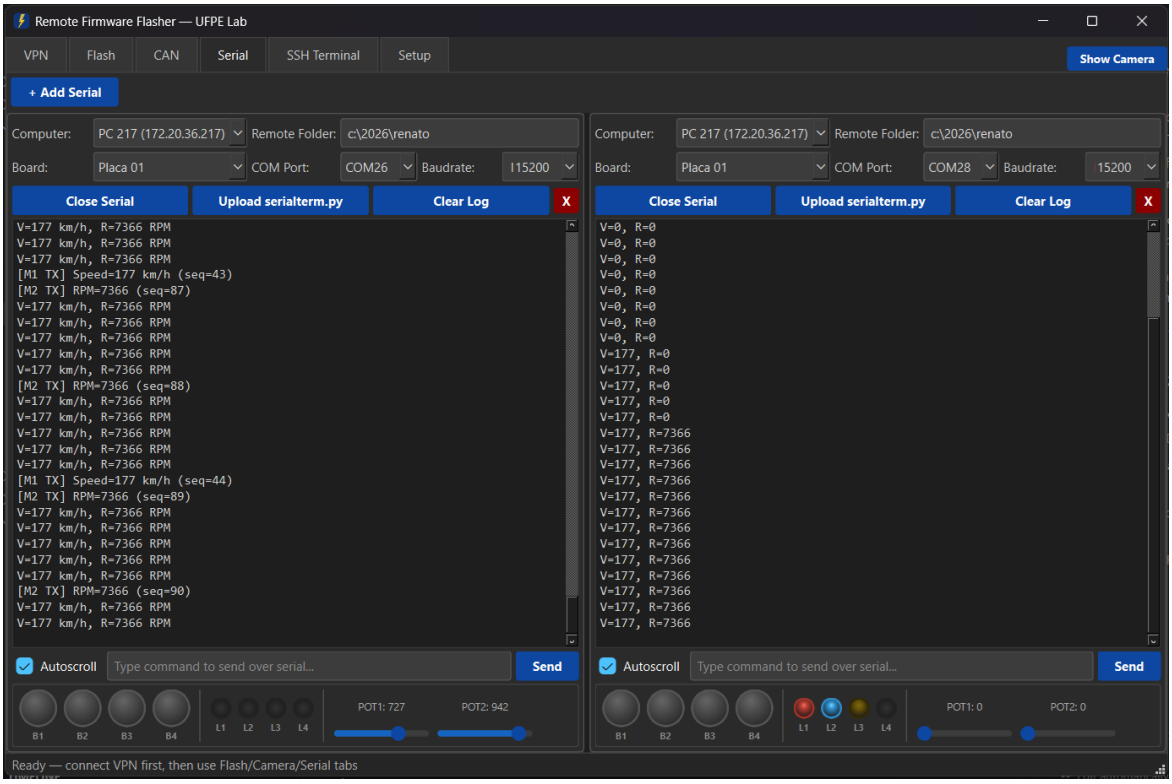

Figura 11: Virtual I/O em ação — potenciômetros controlando velocidade CAN, LEDs indicando TX/RX.

Para utilizar o Virtual I/O no firmware, inclua a biblioteca `VirtualIO.h` (disponível em `arduino_lib/VirtualIO/`). Veja a Seção 21 para detalhes do protocolo e exemplos.

9. Terminal SSH e Upload de Arquivos

A aba **SSH Terminal** combina execução de comandos e upload de arquivos.

9.1 Executar comandos

Digite um comando e pressione Enter. A saída é exibida no log.

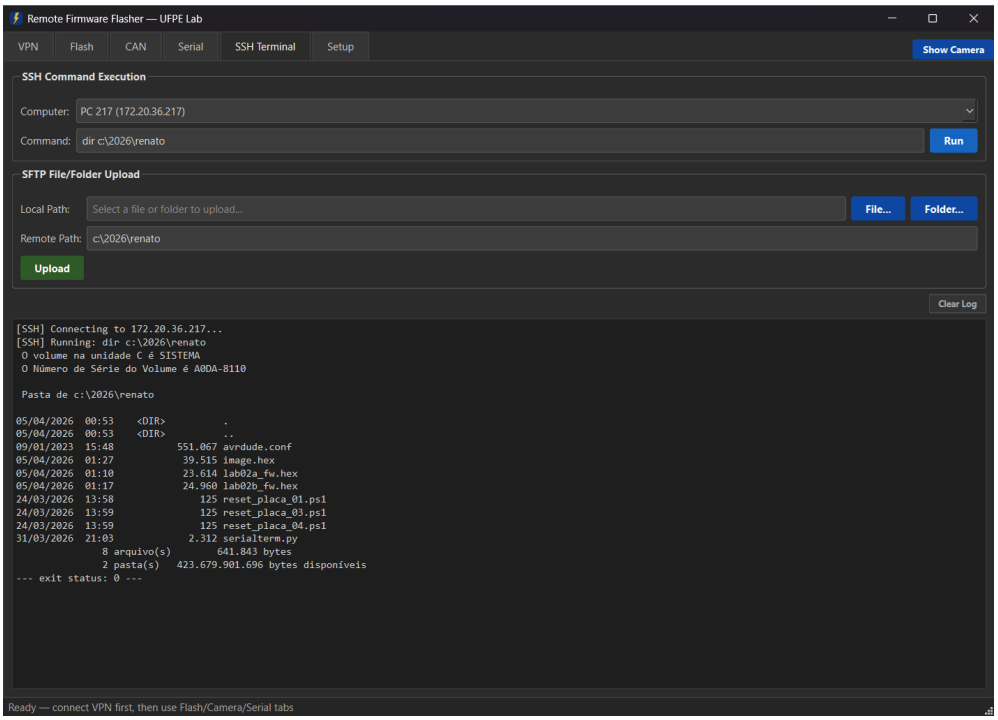

Figura 12: Terminal SSH — listagem de diretório remoto com seção de upload SFTP acima.

Nota

O PC remoto usa Windows. Use comandos como `dir`, `cd`, `type`, e não `ls`, `pwd`, `cat`.

9.2 Upload de arquivos e pastas (SFTP)

1. Clique em **"File..."** para selecionar um arquivo ou **"Folder..."** para uma pasta inteira
2. Defina o caminho remoto de destino
3. Clique em **"Upload"**
4. Acompanhe o progresso na barra

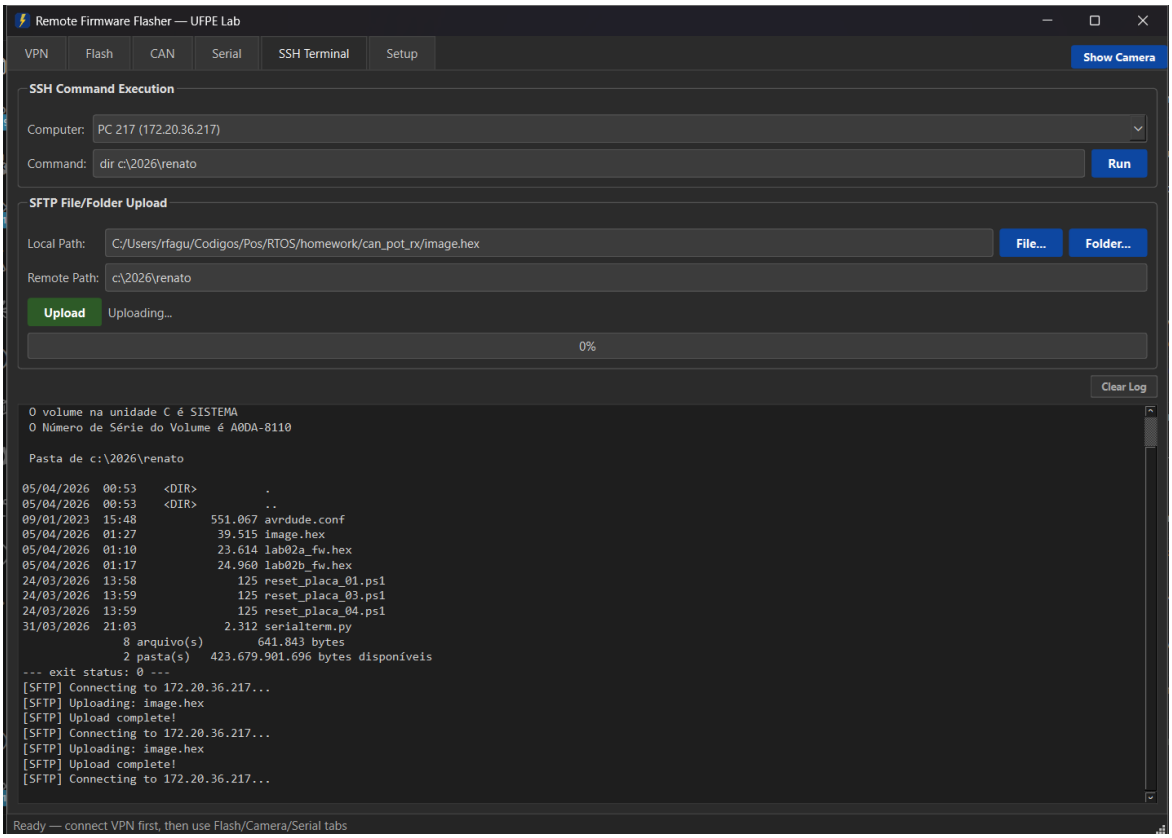

Figura 13: Upload de pasta em andamento — barra de progresso e listagem do diretório remoto.

A transferência usa SFTP (via paramiko) e cria diretórios remotos automaticamente. Pastas são enviadas recursivamente.

10. Câmera ao Vivo

O painel de câmera exibe o feed MJPEG do laboratório. É visível nas abas Flash, Serial e SSH, podendo ser oculto/exibido pelo botão no canto da barra de abas. Com múltiplas conexões serial, a câmera reduz automaticamente para 25% da largura.

11. Integração com Notepad++

A aba **Setup** contém guias completas de integração. Abaixo está um resumo e um exemplo prático.

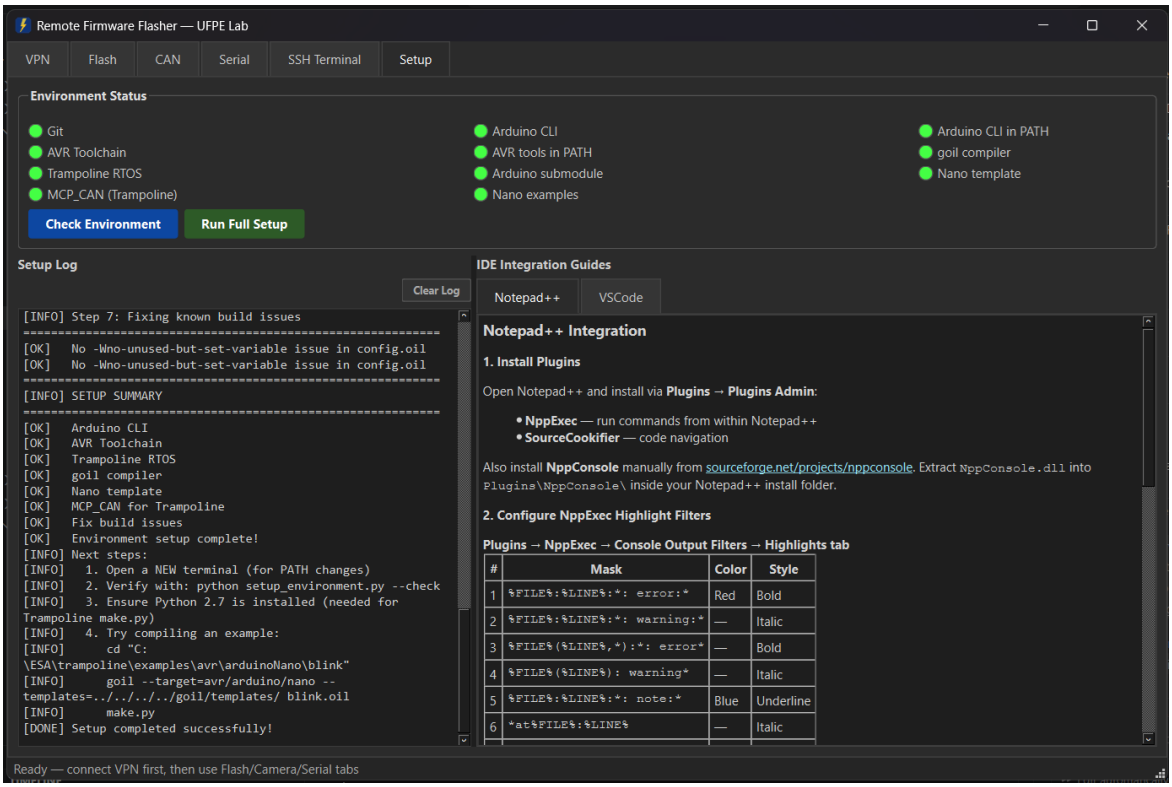

Figura 14: Aba Setup — todos os componentes instalados (verde) e guia do Notepad++ à direita.

11.1 Plugins necessários

Instale via **Plugins** → **Plugins Admin**:

- **NppExec** — execução de comandos e scripts
- **SourceCookifier** — navegação de código

Instale manualmente o **NppConsole** (baixe de https://sourceforge.net/projects/nppconsole/).

11.2 Exemplo: Compilar e gravar um projeto Trampoline

Crie o seguinte script em `Plugins` → `NppExec` → `Execute`:

Listing 1: Script NppExec: trampoline_build_and_flash.scp

```
// [Trampoline] Build and Flash
NPP_SAVEALL
cd $(CURRENT_DIRECTORY)

// Run goil to generate build files
goil --target=avr/arduino/nano --templates=../../../../goil/templates
    / $(NAME_PART).oil

// Compile
cmd /c python make.py

// Flash (change COM port as needed)
```

```
INPUTBOX "Serial port:":"COM": "COM25"
set local commport = $(INPUT)
arduino-cli upload -b arduino:avr:nano:cpu=atmega328old -p $(commport
    ) -v -i image.hex -t
```

Dica

Para criar um atalho de teclado: **Plugins** → **NppExec** → **Advanced Options**, associe o script a um item de menu, depois em **Settings** → **Shortcut Mapper** atribua uma tecla.

11.3 Máscaras de destaque para erros

Configure em **Plugins** → **NppExec** → **Console Output Filters** → **Highlights**:

#	Máscara		Cor	Estilo
1	`%FILE%:%LINE%:*:`	`error:*`	Vermelho	Negrito
2	`%FILE%:%LINE%:*:`	`warning:*`	—	Itálico
3	`%FILE%:%LINE%:*:`	`note:*`	Azul	Sublinhado

12. Integração com VSCode

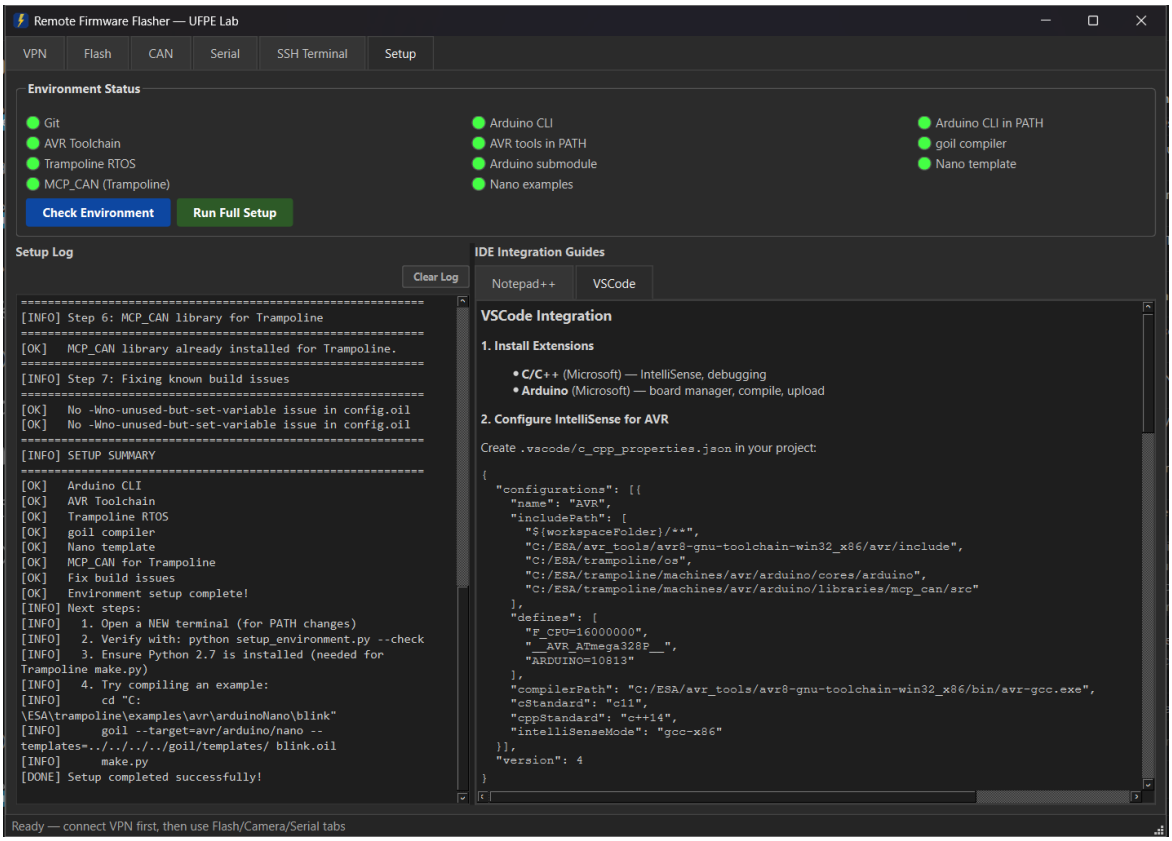

Figura 15: Guia VSCode na aba Setup — configuração do IntelliSense com `c_cpp_properties.json`.

12.1 Extensões recomendadas

- **C/C++** (Microsoft) — IntelliSense e depuração
- **Arduino** (Microsoft) — gerenciamento de placas

12.2 IntelliSense para AVR

Crie o arquivo `.vscode/c_cpp_properties.json` na raiz do seu projeto:

Listing 2: .vscode/c_cpp_properties.json

```
{
  "configurations": [{
    "name": "AVR",
    "includePath": [
      "${workspaceFolder}/**",
      "C:/ESA/avr_tools/avr8-gnu-toolchain-win32_x86/avr/include",
      "C:/ESA/trampoline/os",
      "C:/ESA/trampoline/machines/avr/arduino/cores/arduino",
      "C:/ESA/trampoline/machines/avr/arduino/libraries/mcp_can/src"
    ],
```

```
    "defines": ["F_CPU=16000000", "__AVR_ATmega328P__", "ARDUINO
       =10813"],
    "compilerPath":
      "C:/ESA/avr_tools/avr8-gnu-toolchain-win32_x86/bin/avr-gcc.exe
         ",
    "cStandard": "c11",
    "cppStandard": "c++14",
    "intelliSenseMode": "gcc-x86"
  }],
  "version": 4
}
```

12.3 Tarefas de build (tasks.json)

Crie `.vscode/tasks.json` para automatizar compilação e gravação:

Listing 3: .vscode/tasks.json (resumido)

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Trampoline: goil + make",
      "type": "shell",
      "command": "goil --target=avr/arduino/nano --templates=${input:
         base}/goil/templates/ ${fileBasename} && python make.py",
      "options": { "cwd": "${fileDirname}" },
      "group": { "kind": "build", "isDefault": true },
      "problemMatcher": ["$gcc"]
    },
    {
      "label": "Trampoline: Upload",
      "type": "shell",
      "command": "arduino-cli upload -b arduino:avr:nano:cpu=
         atmega328old -p ${input:com} -v -i image.hex -t",
      "options": { "cwd": "${fileDirname}" },
      "problemMatcher": []
    },
    {
      "label": "Arduino: Compile + Upload",
      "type": "shell",
      "command": "arduino-cli compile -b arduino:avr:nano -v -e . &&
         arduino-cli upload -b arduino:avr:nano -p ${input:com} -v -t
         ",
      "group": "build",
      "problemMatcher": ["$gcc"]
    }
  ],
  "inputs": [
    { "id": "com", "description": "COM port", "default": "COM25", "
       type": "promptString" },
    { "id": "base", "description": "Trampoline base path", "default":
        "../../../..", "type": "promptString" }
  ]
}
```

Dica

Use **Ctrl+Shift+B** para executar a tarefa de build padrão ("goil + make") e **Ctrl+Shift+P → Tasks: Run Task** para as demais.

12.4 Configurações recomendadas

Adicione ao `.vscode/settings.json`:

```
{
  "files.associations": { "*.oil": "c", "*.ino": "cpp" },
  "C_Cpp.errorSquiggles": "enabled",
  "terminal.integrated.defaultProfile.windows": "Git Bash"
}
```

13. Guia de Desenvolvimento — Criando seu Projeto

13.1 Passo a passo

1. Copie uma pasta de exemplo de `examples/` para `C:\ESA\trampoline\opt\devel\<seu_projeto>`
2. Renomeie os arquivos `.oil` e `.cpp` com o nome do seu projeto
3. Edite o `.oil`: atualize `APP_SRC` para seu `.cpp`
4. Edite o `.cpp`: implemente sua lógica
5. Copie `Board.h` (e `VirtualIO.h/cpp` se precisar) para a pasta
6. Compile: `goil -target=avr/arduino/nano -templates=../../../goil/templates/<nome>.oil`
7. Build: `python build.py`
8. O arquivo `image.hex` é gerado na pasta do projeto
9. Grave no Arduino usando o Remote Flasher (aba Flash) ou `arduino-cli`

Importante

Os projetos devem estar dentro de `C:\ESA\trampoline\opt\devel\` para que o caminho relativo `TRAMPOLINE_BASE_PATH = "../../../"` funcione corretamente.

14. Referência Rápida — Arquivo OIL

O arquivo `.oil` descreve a configuração do RTOS em linguagem OIL (OSEK Implementation Language). Abaixo estão os padrões mais usados.

14.1 Estrutura básica

```
OIL_VERSION = "2.5" : "nome_projeto";

CPU test {
  OS config {
    STATUS = STANDARD;
    BUILD = TRUE {
      TRAMPOLINE_BASE_PATH = "../../../";
      APP_NAME = "image";
      APP_SRC = "meu_arquivo.cpp";
      // APP_SRC = "VirtualIO.cpp";  // se usar Virtual I/O
      CPPCOMPILER = "avr-g++";
      COMPILER = "avr-gcc"; LINKER = "avr-gcc";
      ASSEMBLER = "avr-gcc"; COPIER = "avr-objcopy";
      SYSTEM = PYTHON;
      LIBRARY = serial;
      // LIBRARY = spi;      // se usar CAN
      // LIBRARY = mcp_can;  // se usar CAN
    };
    SYSTEM_CALL = TRUE;
  };
  APPMODE stdAppmode {};

  // ... TASKs, ALARMs, RESOURCEs aqui ...
};
```

14.2 TASK — Declaração de tarefa

```
  TASK NomeDaTask {
    PRIORITY = 10;          // maior = mais prioritário
    AUTOSTART = FALSE;      // TRUE se deve iniciar automaticamente
    ACTIVATION = 1;         // máximo de ativações pendentes
    SCHEDULE = FULL;        // FULL = preemptivo, NON = cooperativo
    STACKSIZE = 256;        // tamanho da pilha em bytes
    // RESOURCE = nomeRecurso;  // se usar recurso compartilhado
  };
```

No código C++, a task é declarada com `TASK(NomeDaTask) { ... }`. **Sempre termine com `TerminateTask();`.**

14.3 ALARM — Ativação periódica

```
  ALARM nomeAlarme {
    COUNTER = SystemCounter;
    ACTION = ACTIVATETASK { TASK = NomeDaTask; };
    AUTOSTART = TRUE {
      ALARMTIME = 500;      // primeiro disparo (ms)
      CYCLETIME = 500;      // período de repetição (ms)
      APPMODE = stdAppmode;
    };
  };
```

API para mudar período em tempo de execução:

```
CancelAlarm(nomeAlarme);                    // cancela alarme ativo
SetRelAlarm(nomeAlarme, delay, cycle);      // reprograma (ms)
```

Ver exemplo `03_alarm_dynamic` na pasta `examples/`.

14.4 RESOURCE — Exclusão mútua

```
  RESOURCE nomeRecurso {
    RESOURCEPROPERTY = STANDARD;  // ou INTERNAL
  };
```

Uso no código C++:

```
TASK(MinhaTask) {
    GetResource(nomeRecurso);
    // ... seção crítica (acesso exclusivo) ...
    ReleaseResource(nomeRecurso);
    TerminateTask();
}
```

Declare `RESOURCE = nomeRecurso;` na TASK que o utiliza (no .oil). Recursos impedem inversão de prioridade usando o protocolo *Priority Ceiling*.

Ver exemplo `04_resource_prodcons` na pasta `examples/`.

15. Referência Rápida — Comunicação CAN

15.1 Configuração no arquivo OIL

Para usar CAN, adicione as bibliotecas `spi` e `mcp_can`:

```
      LIBRARY = serial;
      LIBRARY = spi;
      LIBRARY = mcp_can;
```

15.2 Inicialização

```
#include "Board.h"
#include <mcp_can.h>

MCP_CAN CAN1(CAN1_CS);  // chip select da placa

void setup() {
    Serial.begin(115200);
    pinMode(CAN_INT, INPUT);
    while (CAN1.begin(MCP_ANY, CAN_500KBPS, MCP_8MHZ) != CAN_OK) {
        delay(200);
    }
    CAN1.setMode(MCP_NORMAL);
}
```

15.3 Enviar mensagem

```
byte data[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};

// Frame standard (11 bits)
CAN1.sendMsgBuf(0x100, CAN_STDID, 8, data);

// Frame extended (29 bits)  usado em J1939
CAN1.sendMsgBuf(0x18FEF100, CAN_EXTID, 8, data);
```

15.4 Receber mensagem

```
TASK(Receiver) {
    if (!digitalRead(CAN_INT)) {    // há mensagem no buffer?
        unsigned char dlc = 0;
        unsigned char data[8];
        long unsigned int id;

        CAN1.readMsgBuf(&id, &dlc, data);

        unsigned long canId = id & 0x1FFFFFFFUL;  // 29-bit ID
        // data[0..dlc-1] contém os bytes da mensagem
    }
    TerminateTask();
}
```

15.5 Encapsular/Desencapsular sinais

Para encapsular um valor em bytes do frame CAN:

```
// Encapsular: valor float -> bytes CAN com resolução
float velocidade_kmh = 120.5;
float resolucao = 1.0 / 256.0;  // 1/256 km/h por bit
unsigned int raw = (unsigned int)(velocidade_kmh / resolucao);
data[1] = (raw >> 8) & 0xFF;  // byte alto
data[2] = raw & 0xFF;          // byte baixo
```

Para desencapsular:

```
// Desencapsular: bytes CAN -> valor float
unsigned int raw = ((unsigned int)data[1] << 8) | data[2];
float velocidade_kmh = raw * resolucao;
```

Ver exemplos `05_can_basic` e `07_can_signals` na pasta `examples/`.

16. Referência Rápida — Board.h (Pinos da Placa)

O arquivo `Board.h` define os pinos do *Automotive Network Evaluation Board* (ANEB v1.1). Cada ECU possui:

Tipo	Pinos (ECU1)	Função
Entrada digital	`ECU1_DIN1..DIN4`	Pushbuttons (`digitalRead`)
Saída digital	`ECU1_DOUT1, DOUT2`	LEDs (`digitalWrite`)
Buzzer	`ECU1_BUZZER`	Buzzer (só ECU1)
Entrada analógica	`ECU1_AIN1..AIN4`	Potenciômetros (`analogRead`, 0–1023)
Saída analógica	`ECU1_LOOP, LDR_LED`	PWM (`analogWrite`, 0–255, só ECU1)
CAN chip select	`ECU1_CAN1_CS`	MCP2515 SPI CS (`CAN1_CS`)
CAN interrupt	`ECU1_CAN1_INT`	MCP2515 INT (`CAN_INT`)

Para usar outra ECU, substitua o prefixo: `ECU2_DIN1`, `ECU3_AIN2`, etc. A pinagem física é a mesma para todas as ECUs.

Ver exemplo `08_board_io` na pasta `examples/`.

17. Referência Rápida — Variáveis Compartilhadas entre Tasks

Tasks no Trampoline podem compartilhar dados através de variáveis globais. Use `volatile` para garantir que o compilador não otimize o acesso:

```
static volatile int sharedValue = 0;  // compartilhada entre tasks

TASK(Producer) {
    sharedValue = analogRead(ECU1_AIN1);  // escreve
    TerminateTask();
}

TASK(Consumer) {
    int local = sharedValue;  // lê
    Serial.println(local);
    TerminateTask();
}
```

Cuidado com inconsistência

Se uma task de alta prioridade interrompe outra durante a escrita de uma variável multi-byte, o leitor pode ver um valor parcialmente atualizado. Use `RESOURCE` para proteger seções críticas:

```
// No .oil: RESOURCE dataRes { RESOURCEPROPERTY = STANDARD; };

TASK(Writer) {
    GetResource(dataRes);
    sharedSpeed = newSpeed;    // escrita protegida
    sharedRPM = newRPM;
    ReleaseResource(dataRes);
    TerminateTask();
}

TASK(Reader) {
    int s, r;
```

```
    GetResource(dataRes);
    s = sharedSpeed;            // leitura protegida
    r = sharedRPM;
    ReleaseResource(dataRes);
    Serial.print("V="); Serial.print(s);
    Serial.print(" R="); Serial.println(r);
    TerminateTask();
}
```

Ver exemplo `04_resource_prodcons` na pasta `examples/`.

18. Exemplos do Repositório

A pasta `examples/` do repositório contém projetos prontos para compilar. Copie para `C:\ESA\trampoline\opt\devel\` e siga o guia da Seção 12.

#	Exemplo	Conceitos
01	`blink`	Task + Alarm + `TerminateTask()`
02	`multitask`	Múltiplas tasks, prioridades, preempção
03	`alarm_dynamic`	`SetRelAlarm()`, `CancelAlarm()`, mudança de período
04	`prodcons`	`RESOURCE`, `GetResource()`, `ReleaseResource()`
05	`can_basic`	TX + RX básico com MCP2515
06	`virtualio_demo`	Botões, potenciômetros e LEDs virtuais (VirtualIO)
07	`can_signals`	Codificação de sinais, resolução, offset, contador de sequência
08	`board_io`	Leitura de pinos físicos com Board.h (botões, pots, LEDs)

Cada exemplo contém arquivos `.oil` e `.cpp` prontos. Copie `Board.h` para projetos que usam pinos físicos ou CAN, e `VirtualIO.h/cpp` para projetos que usam os controles virtuais.

19. Exemplo Prático: LED Controlado por Alarme

Este exemplo cria uma tarefa no Trampoline que alterna um LED a cada 500 ms usando um alarme do RTOS. **Diferente do blink básico**, este exemplo usa duas tarefas com prioridades distintas e comunicação via evento.

19.1 Estrutura do projeto

```
C:\ESA\trampoline\opt\devel\led_alarm\
led_alarm.oil  # Descrição OSEK do sistema
led_alarm.cpp  # Código-fonte da aplicação
Board.h        # Definições de pinos (copiar da apostila)
```

19.2 Arquivo OIL

Listing 4: led_alarm.oil

```
OIL_VERSION = "2.5" : "led_alarm";

CPU test {
  OS config {
    STATUS = STANDARD;
    BUILD = TRUE {
      TRAMPOLINE_BASE_PATH = "../../../";
      APP_NAME = "image";
      APP_SRC = "led_alarm.cpp";
      CPPCOMPILER = "avr-g++";
      COMPILER = "avr-gcc";
      LINKER = "avr-gcc";
      ASSEMBLER = "avr-gcc";
      COPIER = "avr-objcopy";
      SYSTEM = PYTHON;
      LIBRARY = serial;
    };
    SYSTEM_CALL = TRUE;
  };

  APPMODE stdAppmode {};

  // Alarme que ativa ToggleLed a cada 500ms
  ALARM ledAlarm {
    COUNTER = SystemCounter;
    ACTION = ACTIVATETASK { TASK = ToggleLed; };
    AUTOSTART = TRUE {
      ALARMTIME = 500;
      CYCLETIME = 500;
      APPMODE = stdAppmode;
    };
  };

  // Alarme que ativa PrintStatus a cada 2000ms
  ALARM statusAlarm {
    COUNTER = SystemCounter;
    ACTION = ACTIVATETASK { TASK = PrintStatus; };
    AUTOSTART = TRUE {
      ALARMTIME = 2000;
      CYCLETIME = 2000;
      APPMODE = stdAppmode;
    };
  };

  TASK ToggleLed {
    PRIORITY = 10;
    AUTOSTART = FALSE;
    ACTIVATION = 1;
    SCHEDULE = FULL;
    STACKSIZE = 128;
  };

  TASK PrintStatus {
    PRIORITY = 5;
    AUTOSTART = FALSE;
```

```
    ACTIVATION = 1;
    SCHEDULE = FULL;
    STACKSIZE = 256;
  };
};
```

19.3 Código-fonte

Listing 5: led_alarm.cpp

```
#include "tpl_os.h"
#include "Arduino.h"

static bool ledState = false;
static unsigned long toggleCount = 0;

void setup() {
  Serial.begin(115200);
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.println("LED Alarm example started.");
}

// Task de alta prioridade: alterna o LED
TASK(ToggleLed) {
  ledState = !ledState;
  digitalWrite(LED_BUILTIN, ledState ? HIGH : LOW);
  toggleCount++;
}

// Task de baixa prioridade: imprime status na serial
TASK(PrintStatus) {
  Serial.print("LED toggles: ");
  Serial.print(toggleCount);
  Serial.print(" | State: ");
  Serial.println(ledState ? "ON" : "OFF");
}
```

19.4 Compilar e gravar

Abra um terminal na pasta do projeto e execute:

```
cd C:\ESA\trampoline\opt\devel\led_alarm
goil -target=avr/arduino/nano -templates=../../../goil/templates/
led_alarm.oil
python make.py
arduino-cli upload -b arduino:avr:nano:cpu=atmega328old -p COM25 -v -i
image.hex -t
```

Ou use o Remote Flasher:

1. Copie `image.hex` para sua máquina
2. Na aba Flash, selecione o PC, placa e porta
3. Faça “Upload + Reset + Flash”

4. Na aba Serial, abra a porta para ver o status sendo impresso a cada 2 segundos

20. Exemplo Prático: Comunicação CAN com RTOS

Este exemplo demonstra duas ECUs trocando mensagens CAN usando tarefas do Trampoline e controles **Virtual I/O**. O transmissor usa POT1/POT2 do painel GUI para definir valores e os envia via CAN. O receptor decodifica e mostra o resultado nos LEDs virtuais.

20.1 ECU Transmissora (can_pot_tx)

Listing 6: can_pot_tx.oil (trecho relevante)

```
      APP_SRC = "can_pot_tx.cpp";
      APP_SRC = "VirtualIO.cpp";
      LIBRARY = serial;
      LIBRARY = spi;
      LIBRARY = mcp_can;

  ALARM sendAlarm {
    COUNTER = SystemCounter;
    ACTION = ACTIVATETASK { TASK = SendPotValue; };
    AUTOSTART = TRUE {
      ALARMTIME = 250;
      CYCLETIME = 250;
      APPMODE = stdAppmode;
    };
  };

  TASK SendPotValue {
    PRIORITY = 20;
    AUTOSTART = FALSE;
    ACTIVATION = 1;
    SCHEDULE = FULL;
    STACKSIZE = 256;
  };
```

Listing 7: can_pot_tx.cpp

```
#include "tpl_os.h"
#include "Arduino.h"
#include "Board.h"
#include "VirtualIO.h"
#include <mcp_can.h>

#define POT_ID 0x200
MCP_CAN CAN1(CAN1_CS);

void setup() {
  Serial.begin(115200);
  vioInit();
  while (CAN1.begin(MCP_ANY, CAN_500KBPS, MCP_8MHZ) != CAN_OK) {
    delay(200);
```

```
  }
  CAN1.setMode(MCP_NORMAL);
  Serial.println("CAN TX ready. Use POT1 and POT2.");
}

TASK(SendPotValue) {
  vioUpdate();
  int pot1 = vAnalogRead(VPOT1);
  int pot2 = vAnalogRead(VPOT2);
  byte data[4] = { highByte(pot1), lowByte(pot1),
                   highByte(pot2), lowByte(pot2) };
  byte ret = CAN1.sendMsgBuf(POT_ID, CAN_STDID, 4, data);
  if (ret == CAN_OK) {
    Serial.print("TX pot1=");
    Serial.print(pot1);
    Serial.print(" pot2=");
    Serial.println(pot2);
  }
  TerminateTask();
}
```

20.2 ECU Receptora (can_pot_rx)

Listing 8: can_pot_rx.cpp (trecho relevante)

```
#include "VirtualIO.h"

TASK(ReceiveAndAct) {
  vioUpdate();
  if (!digitalRead(CAN_INT)) {
    unsigned char dlc = 0;
    unsigned char data[8];
    long unsigned int id;
    CAN1.readMsgBuf(&id, &dlc, data);

    if ((id & 0x1FFFFFFF) == POT_ID && dlc >= 4) {
      int pot1 = (data[0] << 8) | data[1];
      int pot2 = (data[2] << 8) | data[3];

      // LEDs virtuais mostram os valores recebidos
      vLedWrite(VLED1, pot1 / 4);  // vermelho = pot1
      vLedWrite(VLED2, pot2 / 4);  // azul = pot2
      vLedWrite(VLED3, 255);       // verde = CAN ativo

      Serial.print("RX pot1=");
      Serial.print(pot1);
      Serial.print(" pot2=");
      Serial.println(pot2);
    }
  }
  TerminateTask();
}
```

Dica

Abra duas janelas seriais no Remote Flasher: TX em um painel (mova POT1/POT2) e RX no outro (observe os LEDs virtuais mudando de brilho). Faça o upload de `serialterm.py` antes de conectar.

21. Virtual I/O — Controles Remotos

Cada painel serial possui um painel de **Virtual I/O** com 4 botões e 2 potenciômetros virtuais. Esses controles enviam comandos pela porta serial, permitindo simular entradas físicas sem acesso à placa.

21.1 Protocolo

Comando	Significado	Exemplo
`!B1:1`	Botão 1 pressionado	Envia ao pressionar BTN1
`!B1:0`	Botão 1 solto	Envia ao soltar BTN1
`!P1:512`	Potenciômetro 1 = 512	Envia ao mover slider POT1
`!P2:0`	Potenciômetro 2 = 0	Faixa: 0–1023

O prefixo `!` diferencia comandos VIO de texto normal. Os sliders possuem *throttle* de 50 ms para evitar sobrecarga na serial.

21.2 LEDs Virtuais (saídas)

Além dos controles de entrada, cada painel possui 4 **LEDs virtuais** (L1–L4) que o Arduino pode controlar. Os LEDs aparecem entre os botões e os potenciômetros, com cores correspondentes (vermelho, azul, amarelo, verde).

O firmware envia comandos pela serial para atualizar os LEDs:

Comando	Significado	Exemplo
`!L1:1`	LED 1 ligado	`vLedWrite(VLED1, 1)`
`!L1:0`	LED 1 desligado	`vLedWrite(VLED1, 0)`
`!L2:128`	LED 2 meio brilho	`vLedWrite(VLED2, 128)`

A função `vLedWrite()` só envia o comando quando o valor muda, evitando sobrecarregar a serial. Os comandos `!L` são interceptados pelo Remote Flasher e **não aparecem no log** — apenas atualizam os indicadores visuais.

21.3 Biblioteca Arduino: VirtualIO

Para usar os controles virtuais, copie os arquivos `VirtualIO.h` e `VirtualIO.cpp` (disponíveis em `arduino_lib/VirtualIO/`) para a pasta do seu projeto. A biblioteca ocupa apenas 6 bytes de RAM e tem overhead desprezível.

Listing 9: API da biblioteca VirtualIO

```
#include "VirtualIO.h"

void setup() {
  Serial.begin(115200);
  vioInit();  // inicializa o sistema de I/O virtual
}

TASK(MinhaTask) {
  vioUpdate();  // lê comandos pendentes da serial (leve, retorna
     imediato)

  // Ler botões virtuais (retorna 0 ou 1)
  if (vRead(VBTN1)) { /* BTN1 está pressionado */ }
  if (vRead(VBTN3)) { /* BTN3 está pressionado */ }

  // Ler potenciômetros virtuais (retorna 01023)
  int valor = vAnalogRead(VPOT1);
}
```

Importante

Chame `vioUpdate()` no início de qualquer task que use os controles virtuais. A função é leve — retorna imediatamente se não há dados na serial.

21.4 Exemplo 1: LED controlado por botão e brilho por potenciômetro

Este exemplo usa o BTN1 virtual para ligar/desligar um LED e o POT1 para controlar o brilho via PWM.

Listing 10: vio_led.oil

```
OIL_VERSION = "2.5" : "vio_led";

CPU test {
  OS config {
    STATUS = STANDARD;
    BUILD = TRUE {
      TRAMPOLINE_BASE_PATH = "../../../";
      APP_NAME = "image";
      APP_SRC = "vio_led.cpp";
      APP_SRC = "VirtualIO.cpp";
      CPPCOMPILER = "avr-g++";
      COMPILER = "avr-gcc";
      LINKER = "avr-gcc";
      ASSEMBLER = "avr-gcc";
      COPIER = "avr-objcopy";
      SYSTEM = PYTHON;
      LIBRARY = serial;
    };
    SYSTEM_CALL = TRUE;
  };

  APPMODE stdAppmode {};
```

```
  ALARM controlAlarm {
    COUNTER = SystemCounter;
    ACTION = ACTIVATETASK { TASK = ControlLed; };
    AUTOSTART = TRUE {
      ALARMTIME = 100;
      CYCLETIME = 100;
      APPMODE = stdAppmode;
    };
  };

  TASK ControlLed {
    PRIORITY = 10;
    AUTOSTART = FALSE;
    ACTIVATION = 1;
    SCHEDULE = FULL;
    STACKSIZE = 256;
  };
};
```

Listing 11: vio_led.cpp

```
#include "tpl_os.h"
#include "Arduino.h"
#include "VirtualIO.h"

#define LED_PIN 3  // PWM pin (physical)

void setup() {
  Serial.begin(115200);
  vioInit();
  pinMode(LED_PIN, OUTPUT);
  Serial.println("VIO LED example ready.");
  Serial.println("BTN1=toggle, POT1=brightness, BTN2=LED2 blink");
}

TASK(ControlLed) {
  vioUpdate();

  if (vRead(VBTN1)) {
    // BTN1 pressed: physical + virtual LED on, brightness from POT1
    int brightness = vAnalogRead(VPOT1) / 4;  // 0-1023 -> 0-255
    analogWrite(LED_PIN, brightness);
    vLedWrite(VLED1, brightness);    // mirror on GUI LED 1
  } else {
    analogWrite(LED_PIN, 0);
    vLedWrite(VLED1, 0);             // GUI LED 1 off
  }

  // BTN2: toggle GUI LED 2 (no physical counterpart)
  vLedWrite(VLED2, vRead(VBTN2) ? 255 : 0);

  // LED 3 shows POT2 as a bar (brightness proportional to pot)
  vLedWrite(VLED3, vAnalogRead(VPOT2) / 4);
}
```

Para testar:

1. Compile e grave o firmware no Arduino

2. Abra a conexão serial no Remote Flasher
3. Pressione **BTN1** — o LED físico acende e o **LED 1** virtual fica vermelho
4. Mova **POT1** — o brilho do LED físico e virtual muda em tempo real
5. Pressione **BTN2** — o **LED 2** virtual (azul) acende
6. Mova **POT2** — o **LED 3** virtual (amarelo) muda de brilho

21.5 Exemplo 2: CAN com controles virtuais

Este exemplo envia mensagens CAN cujo conteúdo é controlado pelos botões e potenciômetros virtuais. A ECU transmissora usa BTN1–BTN4 como flags de status e POT1/POT2 como valores analógicos no frame CAN.

Listing 12: vio_can_tx.cpp (trecho relevante)

```
#include "tpl_os.h"
#include "Arduino.h"
#include "Board.h"
#include "VirtualIO.h"

#define MSG_ID 0x300
MCP_CAN CAN1(CAN1_CS);

void setup() {
  Serial.begin(115200);
  vioInit();
  while (CAN1.begin(MCP_ANY, CAN_500KBPS, MCP_8MHZ) != CAN_OK) {
    delay(200);
  }
  CAN1.setMode(MCP_NORMAL);
  Serial.println("VIO CAN TX ready. Use BTN1-4 and POT1-2.");
}

TASK(SendFrame) {
  vioUpdate();

  byte data[8];
  // Bytes 0-3: button states
  data[0] = vRead(VBTN1);
  data[1] = vRead(VBTN2);
  data[2] = vRead(VBTN3);
  data[3] = vRead(VBTN4);
  // Bytes 4-5: POT1 (high byte, low byte)
  int pot1 = vAnalogRead(VPOT1);
  data[4] = highByte(pot1);
  data[5] = lowByte(pot1);
  // Bytes 6-7: POT2
  int pot2 = vAnalogRead(VPOT2);
  data[6] = highByte(pot2);
  data[7] = lowByte(pot2);

  CAN1.sendMsgBuf(MSG_ID, CAN_STDID, 8, data);

  Serial.print("TX: BTN=");
  Serial.print(data[0]); Serial.print(data[1]);
  Serial.print(data[2]); Serial.print(data[3]);
  Serial.print(" POT1="); Serial.print(pot1);
  Serial.print(" POT2="); Serial.println(pot2);
```

```
}
```

Dica

Abra duas janelas seriais: uma para o TX (onde você controla os botões e pots virtuais) e outra para o RX (onde você vê os dados recebidos via CAN). Isso simula um painel de controle completo sem acesso físico à placa.

21.6 Exemplo 3: Leitura mista (virtual + real)

A biblioteca pode ser usada em conjunto com entradas físicas. Se o aluno tem acesso remoto à placa mas quer *complementar* os sensores físicos com controles virtuais:

Listing 13: Leitura mista: virtual + real

```
TASK(ReadInputs) {
  vioUpdate();

  // Botões: usa virtual se pressionado, senão lê o físico
  bool btn1 = vRead(VBTN1) || digitalRead(ECU1_DIN1);

  // Potenciômetro: soma virtual + real (com clamp)
  int pot = analogRead(ECU1_AIN1) + vAnalogRead(VPOT1);
  if (pot > 1023) pot = 1023;

  Serial.print("btn1="); Serial.print(btn1);
  Serial.print(" pot="); Serial.println(pot);
}
```

22. Configuração dos PCs do Laboratório

PC	IP	Método Flash	Placas
PC 217	172.20.36.217	avrdude	4 (com reset)
PC 218	172.20.36.218	avrdude	4 (sem reset)
PC 220	172.20.36.220	flash.py	4 (com reset)
PC 221	172.20.36.221	avrdude	4 (sem reset)

Cada placa possui 4 portas ECU (COM), uma porta de reset e um script associado. As portas estão documentadas em `src/lab_config.py`.

23. Estrutura do Projeto

```
remote_flasher/
src/
main.py             # Ponto de entrada
settings.py         # Persistência de configurações
workers.py           # Threads (SSH, SCP, SFTP, Serial, Câmera)
widgets.py           # LogWidget, StatusIndicator
main_window.py       # MainWindow, CameraPanel
lab_config.py       # Configuração de placas/portas
serialterm.py       # Script serial bidirecional (remoto)
tabs/
vpn_tab.py      # Aba VPN
flash_tab.py    # Aba Flash
serial_tab.py    # Aba Serial (até 4 conexões)
ssh_tab.py      # Aba SSH + SFTP
setup_tab.py    # Aba Setup + guias IDE
assets/icon.ico
docs/manual.tex        # Este manual
secrets.py            # Credenciais (gitignored)
setup_environment.py  # Script de configuração do ambiente
requirements.txt
build_exe.bat
```

24. Solução de Problemas

24.1 "make.py" abre diálogo de seleção de programa

O Windows não associou arquivos `.py` ao Python. Execute em um **Prompt de Comando** (CMD) como administrador:

```
assoc .py=Python.File
ftype Python.File="C:\Users\%USERNAME%\AppData
\Local\Programs\Python\Python311
\python.exe%1"%*
```

Alternativa: sempre use `python make.py` em vez de `make.py`.

24.2 "goil" não é reconhecido como comando

Abra um **novo** terminal após executar o Setup (as variáveis PATH são atualizadas no registro do Windows mas terminais já abertos não as recebem).

24.3 VPN não conecta

- Verifique suas credenciais do CIn

- Clique em "Setup VPN Profile" para recriar a conexão
- Verifique se outra VPN não está ativa

24.4 Erro ao gravar firmware

- Verifique se a porta COM está correta
- Tente resetar a placa antes de gravar
- Verifique se o arquivo `.hex` é válido

24.5 Terminal serial: linhas quebradas ou caracteres estranhos

- Verifique se o baudrate está correto (115200 para Trampoline)
- O log agora tem word wrap — linhas longas (ex: frames CAN) são exibidas corretamente

24.6 Setup: "Python interpreter not found"

O executável precisa de um Python instalado no sistema para executar o script de setup. Instale Python 3 e certifique-se de marcar "Add to PATH" durante a instalação.

24.7 Setup: download falha com erro SSL

Alguns servidores universitários têm certificados com problemas. O script automaticamente tenta novamente sem verificação SSL. Se ainda falhar, baixe o componente manualmente (o log indica a URL).